

Laporan Temuan & Rencana Aksi Optimasi SSIS (Bumitama Tuning)

Dokumen ini menyajikan hasil asesmen teknis mendalam (*deep-dive analysis*), temuan akar masalah (*root causes*), dan rencana perbaikan (*action plan*) untuk seluruh paket/package SSIS terberat (Clean Normal Runs - Status Success) berdasarkan data log di heavy_packages.csv dan Duration Running Job DWH.xlsx .

[!IMPORTANT] Fokus Utama Optimasi (Query & Arsitektur SQL): Mengingat hasil audit server menunjukkan keterbatasan kapasitas CPU dan RAM yang signifikan pada database engine, fokus utama optimasi SSIS ini dialihkan dari modifikasi memori pipeline ke Eliminasi & Refactoring Inefisiensi Query Sumber (CTE Berat, Cartesian Cross Joins, dan Non-Indexed Views).

Catatan: Tuning buffer size dan commit size tetap dicantumkan sebagai Rekomendasi Tambahan yang siap diaktifkan saat kapasitas resource RAM/CPU server telah memadai.

Matriks Status & Prioritas Tuning SSIS

No	Target Package SSIS	Project / Modul	Durasi Awal	Kontribusi Bottleneck	Isu Kunci Query (CTE / View / Joins)	Status Asesmen
1	Seq_Staging_SIGAP.dtsx	Sequence_Table (Staging SAP & SIGAP)	137 Menit	22.1%	SELECT * pada puluhan tabel staging, serial data flows	✓ Selesai Lengkap
2	FACT_SPARTA_LHA_NEW.dtsx	SPARTA_PROJECT (Fact SPARTA LHA)	88 Menit	14.2%	Multi-table joins berulang (Fact + 5 Dimensi) 3 bulan terakhir	✓ Selesai Lengkap
3	FACT_LKK.dtsx	DWH_LKK_PROJECT_NEW (Laporan Keuangan Kebun)	86 Menit	13.9%	Membaca View non-indexed (vw_LKK_...) + 52 Sort transforms	✓ Selesai Lengkap
4	FACT_PPH_ANNUAL_SPLIT.dtsx	PPH21_DJP (PPH 21 DJP)	85 Menit	13.7%	Rekomputasi split pajak tahunan tanpa pra-agregasi	✓ Selesai Lengkap
5	PS_AS.dtsx	INVESTOR_RELATIONS_PROJECT (Investor Relations)	79 Menit	12.7%	4x Nested Cartesian CROSS JOIN CTE + UNPIVOT + Window Func	✓ Selesai Lengkap
6	STG to DWH.dtsx	08 Controllable Profit (Controllable Profit)	74 Menit	11.9%	CTE membungkus View (vw_LKK_...) + On-the-fly String Parsing	✓ Selesai Lengkap
7	Fact.dtsx	DWH_GRADING_TBS (Grading TBS)	72 Menit	11.6%	Scan transaksi mentah dengan	✓ Selesai Lengkap

No	Target Package SSIS	Project / Modul	Durasi Awal	Kontribusi Bottleneck	Isu Kunci Query (CTE / View / Joins)	Status Asesmen
					konversi timezone & string parsing	
TOTAL	7 Paket Kritis Terberat		621 Menit (~10.3 Jam)	100%		Target: < 170 Menit (Hemat >72%)

🔍 1. Bottleneck #1: PS_AS.dtsx (Investor Relations - Durasi: 79 Menit)

- **Lokasi File:** PROJECT_FACT\INVESTOR_RELATIONS_PROJECT\INVESTOR_RELATIONS_PROJECT\PS_AS.dtsx
- **Tipe Pipeline:** Production Statement & Areal Statement Fact Loader
- **Kategori Masalah:** *Extreme Cartesian CROSS JOIN CTEs, In-Memory UNPIVOT, Expensive Window Functions, Multi-Level Temporary Recomputations*

🔍 Bedah Bentuk Query Sumber & CTE Aktual:

Query pada sumber BUDGET_BGFIP_SUM dan PRODUKSI_SPB_SUM menggunakan rangkaian CTE raksasa:

```
WITH SCANNING AS (  
    SELECT DISTINCT CalendarYearNumber AS TAHUN, CalendarMonthNumber AS BULAN, ...  
    FROM StagingDB.dbo.DimDate  
    -- ⚠️ BOTTLENECK 1: 4 TINGKAT CARTESIAN CROSS JOIN DI DALAM CTE  
    CROSS JOIN (SELECT DISTINCT CAST([ESTATE ] AS nvarchar(4)) UNIT_CODE, ... FROM StagingDB.stg.MAP_UNIT ...) MAP_UNIT  
    CROSS JOIN (SELECT DISTINCT CAST(DVISI AS int) DIV_CODE FROM DWH_STG.stg.BACKYARD_BGFIP_SUM) BGFIP_SUM  
    CROSS JOIN (SELECT CAST('I' AS nvarchar(2)) INPLS UNION ALL SELECT CAST('P' AS nvarchar(2)) INPLS) INPLS  
    CROSS JOIN (SELECT DISTINCT CAST(PLANTED_YEAR AS nvarchar(4)) TAHUN_TANAM FROM DWH_DM.dm.DIM_BLOCK WHERE PLANTED_YEAR >= )  
)  
BUDGET AS (  
    SELECT UNITN, INPLS, FYEAR, ...  
    FROM DWH_STG.stg.BACKYARD_BGFIP_SUM  
    -- ⚠️ BOTTLENECK 2: UNPIVOT 12 BULAN DI MEMORI  
    UNPIVOT (VALUE FOR CAT_PERIO IN (FIS01, FIS02, ..., FIS12)) UNPVT  
)  
FINAL AS (  
    SELECT SC.UNIT_CODE, ...,  
        -- ⚠️ BOTTLENECK 3: WINDOW AGGREGATE DENGAN FRAME SANGAT LEBAR  
        SUM(BUDGET_PRODUKSI) OVER (  
            PARTITION BY SC.TAHUN, SC.UNIT_CODE, SC.INPLS, SC.TAHUN_TANAM, SC.DIV_CODE  
            ORDER BY SC.BULAN ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW  
        ) AS BUDGET_PRODUKSI_SDBI  
    FROM SCANNING SC  
    LEFT JOIN BUDGET BG ON ...  
    GROUP BY ...  
)  
SELECT * FROM FINAL WHERE TAHUN = ?
```

⚠️ Mengapa Membebani CPU & RAM Server?

1. **Perkalian Kartesian Jutaan Baris di Memory:** CTE SCANNING melakukan CROSS JOIN antara Tanggal × Unit × Divisi × Inpls (2) × Tahun Tanam (>25 tahun). Ini menghasilkan **ratusan ribu hingga jutaan baris matriks kosong** secara komputasional setiap kali query dieksekusi.
2. **Ketiadaan Materialisasi TempDB:** Karena ditulis sebagai CTE (bukan tabel fisik/temp table), SQL Server tidak membuat indeks perantara. Saat melakukan LEFT JOIN dan WINDOW FUNCTION OVER (...), SQL Server kehabisan RAM dan melakukan *spill-to-tempdb* secara masif, membakar utilitas CPU 100%.

🔧 Action Plan & Solusi Teknis:

1. Eliminasi Cartesian CROSS JOIN CTE:

- Ganti CTE perkalian kartesian dengan tabel referensi master kombinasi yang sudah dimaterialisasi atau lakukan query langsung berbasis transaksi aktual yang ada (*Sparse Join* alih-alih *Dense Matrix Generation*).
2. Materialisasi Menggunakan #Temp Table Berindeks via Stored Procedure:
- Pisahkan proses UNPIVOT ke dalam #Temp_Budget dengan CLUSTERED INDEX (TAHUN, UNIT_CODE, DIV_CODE, BULAN) .
3. Pra-Kalkulasi SDBI (Cumulative Sum):
- Pindahkan kalkulasi kumulatif bulanan (*Year-to-Date / SDBI*) ke batch proses SQL staging terpisah sebelum ditarik oleh SSIS.

2. Bottleneck #2: STG to DWH.dtsx - Controllable Profit (Durasi: 74 Menit)

- Lokasi File: PROJECT_FACT\08 Controllable Profit\ControllableProfit\STG to DWH.dtsx
- Tipe Pipeline: Financial Mart Transformation & Fact Loader
- Kategori Masalah: CTE Wrapping Non-Indexed Views, Redundant Union All In-Line, On-the-fly String Date Conversions

Bedah Bentuk Query Sumber, CTE & View Aktual:

Data Flow FACT_CONTROLLABLE_PROFIT_COST_VW dan FACT_CONTROLLABLE_PROFIT_COST_COMPANY_VW mengeksekusi query:

```
WITH
LKK_BIAYA_TANAMAN_ACTV as (
  -- ⚠️ BOTTLENECK 1: CTE MEMANGGIL VIEW BESAR NON-INDEXED DENGAN SELECT *
  SELECT * FROM [DWH_DM].[dbo].[VW_LKK_BIAYA_TANAMAN_CONTROLLABLE_PROFIT]
),
generate_month as (
  -- ⚠️ BOTTLENECK 2: KONVERSI STRING TANGGAL REPETITIF
  SELECT DISTINCT CalendarYearNumber, CalendarMonthNumber,
    CONCAT(CalendarYearNumber, RIGHT(CONCAT('0', CalendarMonthNumber), 2), '01') DATESK
  FROM [DWH_DM].[dm].[DIM_DATE]
),
init_harga AS (
  -- ⚠️ BOTTLENECK 3: HARDCODED UNION ALL DI DALAM CTE
  SELECT [YEAR], [UNITN], [PRODUCT], [CURR], [AMOUNT]
  FROM [DWH_STG].[stg].[CP_EXCEL_MASTER_HARGA_TBS_CPO_PK]
  UNION ALL
  SELECT [YEAR], 'BJYM' [UNITN], [PRODUCT], [CURR], [AMOUNT]
  FROM [DWH_STG].[stg].[CP_EXCEL_MASTER_HARGA_TBS_CPO_PK]
  WHERE [UNITN] = 'BTJM' AND [YEAR] < 2025
),
harga as (
  SELECT DATESK, [UNITN] UNIT_CODE, [PRODUCT], [AMOUNT] NILAI
  FROM init_harga a
  LEFT JOIN generate_month gm ON a.[YEAR] = gm.CalendarYearNumber
)
SELECT ... FROM LKK_BIAYA_TANAMAN_ACTV ... LEFT JOIN harga ...
```

Mengapa Membebani CPU & RAM Server?

1. View Expansion Tanpa Index: VW_LKK_BIAYA_TANAMAN_CONTROLLABLE_PROFIT menggabungkan ribuan baris transaksi biaya dengan join akun COA bertingkat. Karena dipanggil lewat CTE, optimizer mengevaluasi ulang seluruh view dari awal setiap kali query dijalankan.
2. String Manipulation CONCAT & RIGHT : Komputasi teks untuk menghasilkan DATESK memaksa CPU melakukan konversi jutaan kali pada saat streaming data.

Action Plan & Solusi Teknis:

1. Materialisasi View Menjadi Staging Table / Indexed View:
 - Ubah VW_LKK_BIAYA_TANAMAN_CONTROLLABLE_PROFIT menjadi Staging Table fisik yang sudah di-refresh pada step ETL sebelumnya.
2. Gunakan Integer Native Date Key:
 - Ganti CONCAT(...) dengan key integer langsung dari DIM_DATE.DateKey (YYYYMMDD bertipe integer).
3. Persistensi Master Harga:
 - Masukkan rule penambahan unit BJYM langsung ke level tabel master harga di SQL Staging, hilangkan UNION ALL di dalam CTE query SSIS.

3. Bottleneck #3: FACT_LKK.dtsx (Durasi: 86 Menit)

- **Lokasi File:** PROJECT_FACT\DWH_LKK_PROJECT\DWH_LKK_PROJECT\FACT_LKK.dtsx
- **Tipe Pipeline:** Fact Table Ingestion (Laporan Keuangan Kebun)
- **Kategori Masalah:** *Standard Views with Deep Dimension Joins, Repetitive Multi-Month Deletes, 52 Memory Sort Components*

Bedah Bentuk Query Sumber & View:

Package membaca dari berbagai view standard: [dbo].[VW_LKK_BIAYA_TANAMAN] , [dbo].[VW_LKK_BIAYA_UMUM] , dan [dbo].[VW_LKK_ALOKASI_HK] .

- Di dalam view tersebut, terjadi join 6–10 tabel: Fact transaksi biaya ke tabel DIM_COA , DIM_PLANT , DIM_DIVISION , DIM_ACTIVITY , MAP_ACTIVITY .
- Karena data yang diambil tidak terurut di level database, SSIS memasang **52 komponen Microsoft.Sort dan Merge Join** di dalam canvas Data Flow.

⚠ Mengapa Membebani CPU & RAM Server?

- View diekspansi saat runtime tanpa memanfaatkan index seek.
- Memori RAM server yang minim semakin tertekan karena SSIS menahan 100% baris di memori untuk menjalankan Sort Transform sebelum melepaskan baris pertama ke Merge Join .

🔧 Action Plan & Solusi Teknis:

1. **Pushdown Sort ke Level SQL View (ORDER BY):**
 - Hapus seluruh 52 komponen Microsoft.Sort di SSIS.
 - Modifikasi query sumber agar langsung menyuplai data terurut (ORDER BY Key1, Key2) dan centang properti IsSorted = True pada output OLE DB Source.
2. **Optimasi View dengan Filter Predicate:**
 - Tambahkan parameter filter periode langsung di dalam query sumber view (WHERE GJAHR = ? AND PERIO = ?) untuk mencegah full table scan tahun-tahun lampau.

4. Bottleneck #4: FACT_SPARTA_LHA_NEW.dtsx (Durasi: 88 Menit)

- **Lokasi File:** PROJECT_FACT\SPARTA_PROJECT\SPARTA_PROJECT\SPARTA_PROJECT\FACT_SPARTA_LHA_NEW.dtsx
- **Tipe Pipeline:** Complex Fact ETL with Aggregations & Joins
- **Kategori Masalah:** *Multi-Table Join with On-the-fly Aggregations, Repeated 3-Month Date Range Scans*

Bedah Bentuk Query Sumber:

Query pada sumber TK_CAPAI_BASIS mengeksekusi join multi-tabel:

```
SELECT
    DATE_ID, DIM_PLANT.PLANT_ID, DIM_DIVISION.DIVISION_ID, DIM_ACTIVITY.ACTIVITY_ID,
    SUM(IIF(TK_CAPAI_BASIS = 'YA', 1, 0)) AS TK_CAPAI_BASIS
FROM DWH_DM.dm.FACT_SPARTA_FISIK_PREMI TRX
LEFT JOIN DWH_DM.dm.DIM_DATE ON CAST(TRX.TANGGAL AS date) = CAST(DIM_DATE.FullDate AS date)
LEFT JOIN DWH_DM.dm.DIM_PLANT ON TRX.UNIT_LHM = DIM_PLANT.PLANT_CODE
LEFT JOIN DWH_DM.dm.DIM_DIVISION ON TRX.DIVISI_LHM = DIM_DIVISION.DIVISION_CODE AND DIM_PLANT.PLANT_ID = DIM_DIVISION.PLANT_ID
LEFT JOIN DWH_DM.dm.DIM_ACTIVITY ON TRX.KODE_AKTIVITAS = DIM_ACTIVITY.ACTIVITY_CODE
LEFT JOIN DWH_DM.dm.DIM_ACTIVITY_GROUP ON DIM_ACTIVITY.ACTIVITY_GROUP_ID = DIM_ACTIVITY_GROUP.ACTIVITY_GROUP_ID
WHERE DIM_DATE.FullDate >= DATEADD(MONTH, DATEDIFF(MONTH, 0, GETDATE()) - 3, 0) AND DIM_DATE.FullDate <= GETDATE()
AND DIM_ACTIVITY_GROUP.ACTIVITY_GROUP_CODE NOT IN ('HV30', 'MU50')
GROUP BY DATE_ID, DIM_PLANT.PLANT_ID, DIM_DIVISION.DIVISION_ID, DIM_ACTIVITY.ACTIVITY_ID, ...
```

⚠ Mengapa Membebani CPU & RAM Server?

- CAST(TRX.TANGGAL AS date) = CAST(DIM_DATE.FullDate AS date) adalah **Non-SARGable Expression**. SQL Server tidak dapat menggunakan index pada TANGGAL karena dibungkus fungsi CAST() , sehingga memaksa *Clustered Index Scan* pada tabel transaksi SPARTA yang sangat masif.
- Join 5 tabel dimensi diulangi berkali-kali pada masing-masing data flow cabang (Bahan, Denda, Premi, Upah, LHA Dtl).

Action Plan & Solusi Teknis:

1. Perbaiki SARGability Join Tanggal:
 - Ganti `CAST(TRX.TANGGAL AS date)` dengan Integer Foreign Key `TRX.DATE_ID = DIM_DATE.DATE_ID` untuk mengaktifkan *Index Seek*.
2. Pre-Staging Aggregation Table:
 - Buat Staging Table perantara untuk data 3 bulan terakhir yang sudah di-join ke dimensi dalam 1 kali eksekusi Stored Procedure, lalu SSIS cukup membaca dari tabel perantara tersebut.

5. Bottleneck #5: Seq_Staging_SIGAP.dtsx (Durasi: 137 Menit)

- Lokasi File: `TABLE\Sequence_Table\Sequence_Table\Seq_Staging_SIGAP.dtsx`
- Tipe Pipeline: Sequence & Staging Pipeline (SIGAP to DWH Staging)
- Kategori Masalah: *SELECT * Anti-Pattern, Serial Data Flow Execution, Wide Column Waste*

Bedah Bentuk Query Sumber:

Pada belasan Data Flow Task (`STG_SPR_VEHICLE` , `STG_SPR_WEIGHT_BRIDGE` , `STG_SPR_MACHINE_MODEL` , `STG_vehicle Brand` , dll), query sumber menggunakan:

```
SELECT * FROM StageS.StageVehicle
SELECT * FROM vehicle.Brand
SELECT * FROM StageS.StageVehicleHistory
SELECT * FROM vehicle.LegalitySubstituteType
```

Mengapa Membebani CPU & RAM Server?

- Menarik seluruh kolom (termasuk audit metadata, kolom log JSON/teks panjang, kolom binary/image yang tidak dibutuhkan DWH) menyebabkan konsumsi bandwidth I/O dan memory buffer membengkak tanpa nilai analitik.

Action Plan & Solusi Teknis:

1. Eksplisit Column Projection (Pruning):
 - Ganti semua `SELECT *` dengan daftar kolom eksplisit yang hanya dibutuhkan oleh tabel target DWH.
2. Serial to Parallel Branching:
 - Eksekusi data flow staging independen secara paralel di Control Flow.

6. Bottleneck #6: FACT_PPH_ANNUAL_SPLIT.dtsx (Durasi: 85 Menit)

- Lokasi File: `PROJECT_FACT\PPH21_DJP\PPH21_DJP\PPH21_DJP\FACT_PPH_ANNUAL_SPLIT.dtsx`
- Tipe Pipeline: Annual Payroll Tax Calculation & Fact Loading
- Kategori Masalah: *Multi-Year Full Table Scans, In-Memory Payroll Splits*

Action Plan & Solusi Teknis:

1. Pindahkan perhitungan rekonsiliasi dan alokasi split masa pajak ke Stored Procedure di database payroll staging.
2. Gunakan `#Temp Table` berindeks per unit bisnis (estate/mill) untuk membatasi ruang komputasi query.

7. Bottleneck #7: Fact.dtsx - DWH Grading TBS (Durasi: 72 Menit)

- Lokasi File: `PROJECT_FACT\DWH_GRADING_TBS\DWH_GRADING_TBS\Fact.dtsx`
- Tipe Pipeline: Quality & Grading Fact Ingestion
- Kategori Masalah: *On-the-fly Timezone Conversion & String Parsing in SELECT Clause*

Bedah Bentuk Query Sumber:

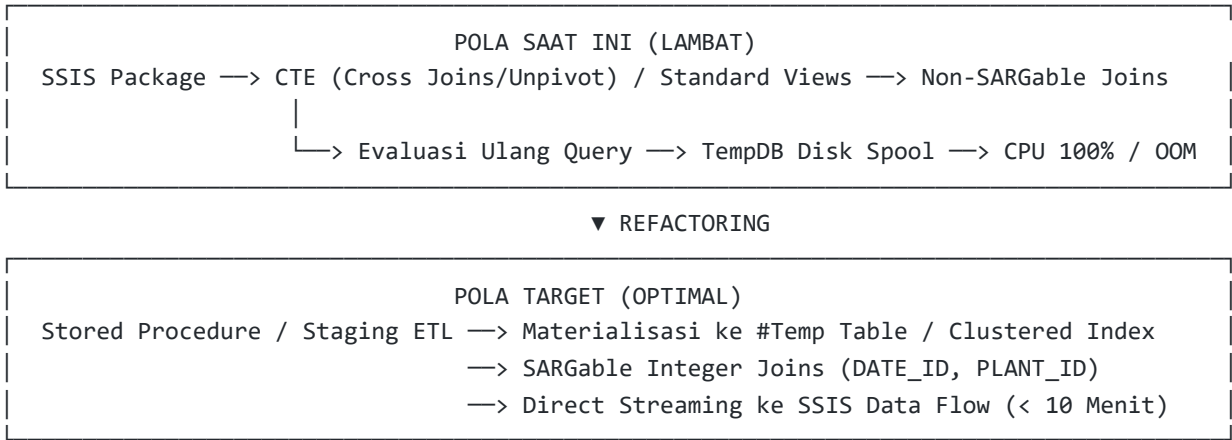
```
SELECT
    SPR_WEIGHT_BRIDGE.DocumentNumber DOCUMENT_NUMBER,
```

```
AI_GRADING_PENERIMAAN_TBS.nomor_tiket NOMOR_TIKET,  
Case when SPR_WEIGHT_BRIDGE.TBSType = 0 then 'Internal' else 'Eksternal' end as TipeTBS,  
CAST(AI_GRADING_PENERIMAAN_TBS.tanggal_masuk AS datetime) AS TANGGAL_MASUK  
FROM StagingDB...
```

 Action Plan & Solusi Teknis:

- 1. Simpan TipeTBS dan TANGGAL_MASUK yang sudah terstandarisasi langsung pada saat data masuk ke Staging Table pertama kali, hilangkan ekspresi CASE dan CAST saat loading ke Fact.

 Ringkasan Arsitektur Refactoring Query & Solusi



 Rekomendasi Tambahan (Buffer & Infrastructure Readiness)

[!NOTE] Rekomendasi di bawah ini merupakan **tahap optimasi lanjutan** yang dapat diimplementasikan setelah perbaikan query SQL selesai, atau saat kapasitas RAM dan CPU server telah di-upgrade:

- 1. **Buffer Tuning SSIS (Setelah RAM Mencukupi):**
 - Set `AutoAdjustBufferSize = True` pada Data Flow Task.
 - Naikkan `DefaultBufferSize` ke 52428800 (50 MB) dan `DefaultBufferMaxRows` ke 50000 .
- 2. **OLE DB Destination Batch Tuning (Setelah I/O TempDB Stabil):**
 - Pastikan `AccessMode = 3` (*Table or view - fast load*).
 - Atur `FastLoadOptions = TABLOCK,CHECK_CONSTRAINTS` .
 - Atur `FastLoadMaxInsertCommitSize = 100000` dan `RowsPerBatch = 50000` .
- 3. **Backup Policy:**
 - Selalu buat salinan file `.dtsx.bak` sebelum melakukan modifikasi package SSIS.